

Санкт-Петербургский политехнический университет Петра Великого
Институт машиностроения, материалов и транспорта
Высшая школа автоматизации и робототехники

КУРСОВАЯ РАБОТА

**Система визуализации инерциальной навигации на основе
полётного контроллера Pixhawk 2.4.8**
по дисциплине «Объектно-ориентированное программирование»

Выполнил студент
гр. 3331506/30401

Томилов А. С.

Руководитель
Доцент

Ананьевский М. С.

«____» _____ 2026г.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Введение	4
Глава 1. Первичное считывание данных с Pixhawk	6
1.1. Протокол MAVLink и сообщение RAW_IMU	6
1.2. Модуль первичного считывания данных	6
1.3. Результаты первичного считывания	7
Глава 2. Архитектура системы визуализации	8
2.1. Общая структура системы визуализации	8
2.2. Структура файлов проекта	8
2.3. Разделяемые структуры данных (imu_shared.h)	9
Глава 3. Зависимости и система сборки	11
3.1. Используемые библиотеки	11
3.2. Конфигурация CMake	11
Глава 4. Математика инерциальной навигации	12
4.1. Калибровка датчиков	12
4.2. Представление ориентации кватернионами	12
4.3. Кватернионное интегрирование гироскопа	12
4.4. Матрица поворота из кватерниона	13
4.5. Углы Эйлера (конвенция ZYX)	13
4.6. Двойное интегрирование (скорость и позиция)	13
4.7. Метод ZUPT	14
Глава 5. Реализация навигационного алгоритма	15
5.1. Функция <code>integrate_imu()</code>	15
5.2. Вычисление углов Эйлера	17
Глава 6. Визуализация	18
6.1. Шейдеры OpenGL 3.3	18
6.2. Процедурная модель полётного контроллера	18
6.3. Главный цикл рендера	19
6.4. HUD-панель графиков	20
Глава 7. Инструкция по установке и запуску	21
7.1. Установка зависимостей	21
7.2. Сборка проекта	21
7.3. Настройка прав и запуск	21
7.4. Управление в окне визуализации	22
Глава 8. Результаты работы	23
8.1. Внешний вид программы	23
8.2. Результаты оценки ориентации	26

8.3. Результаты оценки позиции	27
8.4. Анализ причин дрейфа	27
8.5. Возможные направления развития	27
Заключение	28
Список использованных источников	29

ВВЕДЕНИЕ

Актуальность исследования заключается в том, что инерциальная навигация является одним из ключевых методов определения положения и ориентации объектов в пространстве без использования внешних источников сигналов. Это особенно важно для автономных летательных аппаратов, функционирующих в условиях отсутствия или недоступности спутниковых навигационных систем. Современные полётные контроллеры, такие как Pixhawk, оснащены инерциальными измерительными модулями (IMU — *Inertial Measurement Unit*), позволяющими получать данные об ускорениях и угловых скоростях в реальном времени. Однако обработка этих данных сопряжена с проблемами накопления ошибок интегрирования, что делает задачу повышения точности и устойчивости вычислений актуальной.

Новизна работы заключается в разработке программной системы, объединяющей сбор сырых IMU-данных по протоколу MAVLink, их обработку с применением кватернионного интегрирования и методов компенсации дрейфа, а также визуализацию результатов в реальном времени в интерактивной 3D-среде.

Цель исследования: Разработка программной системы для считывания, обработки и визуализации данных инерциальной навигации с полётного контроллера Pixhawk в реальном времени.

Объект исследования: Процесс инерциальной навигации автономных летательных аппаратов на основе данных IMU.

Предмет исследования: Методы обработки и интегрирования данных акселерометра и гироскопа, а также алгоритмы компенсации накопленных ошибок.

Задачи работы:

1. Реализовать модуль первичного считывания данных по протоколу MAVLink через последовательный порт с записью в CSV.
2. Разработать многопоточную архитектуру приложения.
3. Реализовать алгоритм калибровки и кватернионного интегрирования данных с гироскопа.
4. Реализовать алгоритм двойного интегрирования акселерометра для оценки позиции.
5. Создать OpenGL-визуализацию с 3D-моделью полетного контроллера, тра-

екторией движения и HUD-панелью графиков датчиков.

6. Провести анализ полученных результатов.

Гипотеза исследования: Использование кватернионного интегрирования совместно с методами коррекции, такими как ZUPT, позволит снизить накопление ошибок и повысить точность оценки положения и ориентации объекта.

Практическая значимость заключается в том, что разработанная система может быть использована для тестирования и отладки алгоритмов инерциальной навигации, а также в образовательных и исследовательских целях при изучении работы IMU и систем управления беспилотными аппаратами.

ГЛАВА 1. ПЕРВИЧНОЕ СЧИТЫВАНИЕ ДАННЫХ С PIXHAWK

1.1. Протокол MAVLink и сообщение RAW_IMU

MAVLink (*Micro Air Vehicle Link*) — лёгкий бинарный протокол сериализации для беспилотных систем, разработанный Лоренцем Майером в 2009 году [9]. Протокол широко используется в экосистемах PX4 и ArduPilot [11]. Для получения сырых показаний датчиков применяется сообщение RAW_IMU (идентификатор 27 в диалекте `common`), содержащее девять целочисленных полей в единицах внутренней шкалы датчика (LSB — *Least Significant Bit*):

Таблица 1.1

Поля сообщения MAVLink RAW_IMU

Поле	Датчик	Описание
xacc, yacc, zacc	Акселерометр	Ускорение по осям X, Y, Z (LSB)
xgyro, ygyro, zgyro	Гироскоп	Угловая скорость по осям (LSB)
xmag, ymag, zmag	Магнитометр	Магнитное поле по осям (LSB)

1.2. Модуль первичного считывания данных

На первом этапе работы был написан автономный модуль `raw_data.cpp`, позволивший проверить корректность соединения с Pixhawk и собрать сырые данные для анализа и подбора калибровочных коэффициентов. Взаимодействие реализовано через последовательный порт `/dev/ttyACM0` на скорости 115 200 бод. Инициализация порта выполнена через POSIX-интерфейс `termios` [5]: 8 бит данных, без паритета, один стоп-бит, без аппаратного управления потоком, таймаут 1 с.

Листинг 1.1. Настройка последовательного порта через `termios`

```

1 int setup_serial_port(const char* port, int baudrate) {
2     int fd = open(port, O_RDWR | O_NOCTTY);
3     struct termios tty;
4     memset(&tty, 0, sizeof(tty));
5     tcgetattr(fd, &tty);

```

```

6      cfsetospeed(&tty, baudrate);
7      cfsetispeed(&tty, baudrate);
8      tty.c_cflag |= (CLOCAL | CREAD);
9      tty.c_cflag &= ~CSIZE;
10     tty.c_cflag |= CS8;
11     tty.c_cflag &= ~(PARENB | CSTOPB | CRTSCTS);
12     tty.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG);
13     tty.c_iflag &= ~(IXON | IXOFF | IXANY);
14     tty.c_oflag &= ~OPOST;
15     tty.c_cc[VMIN] = 0;
16     tty.c_cc[VTIME] = 10;
17     tcsetattr(fd, TCSANOW, &tty);
18     return fd;
19 }

```

MAVLink-парсер реализован в виде конечного автомата.

`mavlink_parse_char()` возвращает ненулевое значение только при получении полного валидного пакета с проверкой CRC-16 [10]. Каждый принятый пакет RAW_IMU записывается в CSV-файл с меткой времени с точностью до микросекунды (`gettimeofday()`); принудительный сброс буфера выполняется каждые 50 пакетов.

1.3. Результаты первичного считывания

Сообщения RAW_IMU стабильно поступали с частотой порядка 100 Гц, что соответствует штатной конфигурации IMU на Pixhawk [11]. Сырые данные акселерометра в покое показали устойчивое значение по оси Z (проекция силы тяжести), а гироскоп — дрейф нуля, подлежащий компенсации на следующем этапе.

ГЛАВА 2. АРХИТЕКТУРА СИСТЕМЫ ВИЗУАЛИЗАЦИИ

2.1. Общая структура системы визуализации

На основе опыта, полученного при реализации модуля чтения, была разработана полноценная система **Pixhawk IMU Visualizer** — многопоточное приложение на C++. Два параллельных потока взаимодействуют через потокобезопасный разделяемый буфер (`SharedBuffer`):

Главный поток (`pixhawk_imu_reader.cpp`) устанавливает соединение с Pixhawk по UART, непрерывно парсит входящий байтовый поток и при получении пакета RAW_IMU помещает данные в очередь, а также дублирует их в CSV-файл.

Поток визуализации (`imu_visualizer.cpp`) извлекает пакеты из очереди, запускает алгоритм инерциальной навигации и обновляет 3D-сцену на каждом кадре.

2.2. Структура файлов проекта

Таблица 2.1

Файлы проекта

Файл	Назначение
<code>imu_shared.h</code>	Общие структуры данных: RawIMU, NavState, SharedBuffer, константы калибровки
<code>imu_visualizer.h</code>	Объявление функции <code>run_visualizer()</code>
<code>imu_visualizer.cpp</code>	Навигационный алгоритм, GLSL-шейдеры, геометрия модели и OpenGL-рендер
<code>pixhawk_imu_reader.cpp</code>	MAVLink-ридер, запуск потока визуализации
<code>raw_data.cpp</code>	Модуль первичного считывания
<code>CMakeLists.txt</code>	Система сборки CMake
<code>c_library_v2/</code>	Заголовочная библиотека MAVLink C [9]

2.3. Разделяемые структуры данных (`imu_shared.h`)

Вся общая информация инкапсулирована в трёх структурах. Структура `RawIMU` хранит одно необработанное измерение:

Листинг 2.2. Структуры `RawIMU` и `NavState`

```

1 struct RawIMU {
2     double timestamp;
3     int16_t xacc, yacc, zacc; // acceleration (LSB)
4     int16_t xgyro, ygyro, zgyro; // gyroscope (LSB)
5     int16_t xmag, ymag, zmag; // magnetometer (LSB)
6 };
7
8 struct NavState {
9     double qw=1, qx=0, qy=0, qz=0; // quaternion
10    orientation
11    double wx=0, wy=0, wz=0; // angular velocity
12    (rad/s)
13    double vx=0, vy=0, vz=0; // velocity (m/s)
14    double px=0, py=0, pz=0; // position (m)
15    double roll=0, pitch=0, yaw=0; // Euler angles (
16    deg)
17    double ax=0, ay=0, az=0; // acceleration (m/s
18    ^2)
19 };

```

Структура `SharedBuffer` объединяет всё межпоточное взаимодействие. Каждое поле защищено отдельным мьютексом, что позволяет потокам работать параллельно без глобальной блокировки. Флаг `running` объявлен как `std::atomic<bool>`:

Листинг 2.3. Потокобезопасный разделяемый буфер

```

1 struct SharedBuffer {
2     std::mutex          mtx;
3     std::deque<RawIMU>  queue;
4     static const int    MAX_QUEUE = 500;

```

```

5      std::atomic<bool>      running{true};
6
7      std::mutex              nav_mtx;
8      NavState                nav;
9
10     std::mutex               trail_mtx;
11     std::deque<std::array<float,3>> trail;
12     static const int         MAX_TRAIL = 2000;
13
14     void push(const RawIMU& r) {
15         std::lock_guard<std::mutex> lk(mtx);
16         if ((int)queue.size() < MAX_QUEUE) queue.
17             push_back(r);
18     }
19
20     bool pop(RawIMU& out) {
21         std::lock_guard<std::mutex> lk(mtx);
22         if (queue.empty()) return false;
23         out = queue.front();
24         queue.pop_front();
25         return true;
26     };

```

ГЛАВА 3. ЗАВИСИМОСТИ И СИСТЕМА СБОРКИ

3.1. Используемые библиотеки

Таблица 3.1

Внешние зависимости проекта

Библиотека	Версия	Роль в проекте
MAVLink <code>c_library_v2</code> [9]	2.0	Парсинг телеметрии, декодирование RAW_IMU
GLFW3 [8]	≥ 3.3	Создание окна OpenGL, ввод мыши и клавиатуры
GLEW	≥ 2.1	Загрузка расширений OpenGL 3.3
GLM	$\geq 0.9.9$	Матричная и кватернионная математика
pthread	POSIX	Межпоточная синхронизация (<code>std::thread</code> , <code>std::mutex</code>)

Библиотека MAVLink является заголовочной — она полностью состоит из `.h`-файлов и не требует компиляции [10]. Остальные зависимости устанавливаются из apt-репозитория.

3.2. Конфигурация CMake

Сборочная система использует CMake версии не ниже 3.10 и настроена на стандарт C++14. Зависимости ищутся через `find_package` и `PkgConfig`; проект компилируется с флагами `-O2 -Wall -Wextra`. CMake ищет GLFW тремя способами последовательно: через `find_package`, через `PkgConfig` и, наконец, вручную через `find_path`/`find_library` — это обеспечивает совместимость с различными дистрибутивами GNU Linux.

ГЛАВА 4. МАТЕМАТИКА ИНЕРЦИАЛЬНОЙ НАВИГАЦИИ

4.1. Калибровка датчиков

Сырые показания МЭМС-датчиков содержат постоянное смещение нуля (bias), обусловленное технологическими особенностями изготовления кристалла [6]. Смещения определялись экспериментально по данным CSV-файла в состоянии покоя. Перевод в физические единицы выполняется линейным масштабированием:

$$a_i = (\tilde{a}_i - b_{a,i}) \cdot S_a, \quad \omega_i = (\tilde{\omega}_i - b_{\omega,i}) \cdot S_\omega, \quad (4.1)$$

где $\tilde{a}_i, \tilde{\omega}_i$ — сырые целочисленные отсчёты; $b_{a,i}, b_{\omega,i}$ — смещения нуля; $S_a = 9,81/3050$ [м/с²/отсчёт], $S_\omega = 0,001$ [рад/с/отсчёт].

Экспериментально определённые смещения, вынесенные в пространство имён Calib в файле `imu_shared.h`:

$$\mathbf{b}_a = (31, 24, 0)^\top, \quad \mathbf{b}_\omega = (-7, 1,5, 3)^\top. \quad (4.2)$$

4.2. Представление ориентации кватернионами

Ориентация тела в пространстве описывается единичным кватернионом [2]:

$$\mathbf{q} = q_w + q_x \mathbf{i} + q_y \mathbf{j} + q_z \mathbf{k}, \quad \|\mathbf{q}\| = 1. \quad (4.3)$$

Кватернионы предпочтительнее матриц вращения и углов Эйлера: они не страдают от шарнирного замка (*gimbal lock*), компактны (4 параметра против 9) и численно стабильны при нормализации [2].

4.3. Кватернионное интегрирование гироскопа

Кинематическое уравнение связывает производную кватерниона с угловой скоростью $\boldsymbol{\omega} = (\omega_x, \omega_y, \omega_z)^\top$ в системе координат тела [4]:

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \begin{pmatrix} 0 \\ \boldsymbol{\omega} \end{pmatrix}. \quad (4.4)$$

Для численного интегрирования на шаге Δt применяется точная формула

через ось-угол [6]:

$$\theta = \|\boldsymbol{\omega}\| \cdot \Delta t, \quad \Delta \mathbf{q} = \left(\cos \frac{\theta}{2}, \frac{\boldsymbol{\omega}}{\|\boldsymbol{\omega}\|} \sin \frac{\theta}{2} \right). \quad (4.5)$$

Обновление кватерниона с нормализацией:

$$\mathbf{q} \leftarrow \frac{\mathbf{q} \otimes \Delta \mathbf{q}}{\|\mathbf{q} \otimes \Delta \mathbf{q}\|}. \quad (4.6)$$

4.4. Матрица поворота из кватерниона

Для перевода вектора ускорения из системы координат тела в мировую используется матрица вращения $\mathbf{R} \in SO(3)$ [2]:

$$\mathbf{R} = \begin{pmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y-q_wq_z) & 2(q_xq_z+q_wq_y) \\ 2(q_xq_y+q_wq_z) & 1-2(q_x^2+q_z^2) & 2(q_yq_z-q_wq_x) \\ 2(q_xq_z-q_wq_y) & 2(q_yq_z+q_wq_x) & 1-2(q_x^2+q_y^2) \end{pmatrix}. \quad (4.7)$$

4.5. Углы Эйлера (конвенция ZYX)

Из кватерниона вычисляются углы крена ϕ , тангажа ϑ и рыскания ψ [6]:

$$\phi = \text{atan2}(2(q_wq_x + q_yq_z), 1 - 2(q_x^2 + q_y^2)), \quad (4.8)$$

$$\vartheta = \arcsin(2(q_wq_y - q_zq_x)), \quad (4.9)$$

$$\psi = \text{atan2}(2(q_wq_z + q_xq_y), 1 - 2(q_y^2 + q_z^2)). \quad (4.10)$$

4.6. Двойное интегрирование (скорость и позиция)

Ускорение переводится из системы тела в мировую, из него вычитается гравитация [1]:

$$\mathbf{a}_{\text{world}} = \mathbf{R} \mathbf{a}_{\text{body}} - \mathbf{g}, \quad \mathbf{g} = (0, 0, -9,81)^\top [\text{м/с}^2]. \quad (4.11)$$

Интегрирование методом Эйлера первого порядка:

$$\mathbf{v}(t+\Delta t) = \mathbf{v}(t) + \mathbf{a}_{\text{world}}(t) \cdot \Delta t, \quad (4.12)$$

$$\mathbf{p}(t+\Delta t) = \mathbf{p}(t) + \mathbf{v}(t) \cdot \Delta t. \quad (4.13)$$

4.7. Метод ZUPT

ZUPT (*Zero Velocity Update*) — классический способ частичной коррекции в моменты покоя [12]. Если норма угловой скорости опускается ниже порогового значения $\omega_{\text{th}} = 0,015$ рад/с, скорость мягко гасится с коэффициентом $\alpha = 0,85$:

$$\text{если } \|\boldsymbol{\omega}\| < \omega_{\text{th}} : \quad \mathbf{v} \leftarrow \mathbf{v} \cdot \alpha. \quad (4.14)$$

Дополнительно, если компонента мирового ускорения по модулю меньше $0,3 \text{ м/с}^2$, она обнуляется полностью, что устраняет интегрирование шумового сигнала акселерометра в статике.

ГЛАВА 5. РЕАЛИЗАЦИЯ НАВИГАЦИОННОГО АЛГОРИТМА

5.1. Функция `integrate_imu()`

Весь навигационный алгоритм сосредоточен в функции `integrate_imu()` файла `imu_visualizer.cpp`. Она вызывается из потока визуализации для каждого пакета, извлечённого из очереди. Шаг интегрирования Δt вычисляется как разность временны́х меток двух последовательных пакетов.

Блок 1. Калибровка сырых отсчётов по формуле (4.1):

Листинг 5.4. Калибровка сырых показаний

```

1 double ax = (raw.xacc - Calib::AX_BIAS) * Calib::
    ACCEL_SCALE;
2 double ay = (raw.yacc - Calib::AY_BIAS) * Calib::
    ACCEL_SCALE;
3 double az = (raw.zacc - Calib::AZ_BIAS) * Calib::
    ACCEL_SCALE;
4 double gx = (raw.xgyro - Calib::GX_BIAS) * Calib::
    GYRO_SCALE;
5 double gy = (raw.ygyro - Calib::GY_BIAS) * Calib::
    GYRO_SCALE;
6 double gz = (raw.zgyro - Calib::GZ_BIAS) * Calib::
    GYRO_SCALE;
```

Блок 2. Кватернионное интегрирование через ось-угол по формулам (4.5)–(4.6):

Листинг 5.5. Кватернионное интегрирование гироскопа

```

1 double theta = std::sqrt(gx*gx + gy*gy + gz*gz) * dt;
2 if (theta > 1e-10) {
3     double norm_w = std::sqrt(gx*gx + gy*gy + gz*gz);
4     double s      = std::sin(theta * 0.5) / norm_w;
5     double c      = std::cos(theta * 0.5);
6     double dqw = c,    dqx = gx*s, dqy = gy*s, dqz = gz
        *s;
7
8     // q_new = q (x) dq
```

```

9      double qw = n.qw*dqw - n.qx*dqx - n.qy*dqy - n.qz*
      dqz;
10     double qx = n.qw*dqx + n.qx*dqw + n.qy*dqz - n.qz*
      dqy;
11     double qy = n.qw*dqy - n.qx*dqz + n.qy*dqw + n.qz*
      dqx;
12     double qz = n.qw*dqz + n.qx*dqy - n.qy*dqx + n.qz*
      dqw;
13
14     double norm = std::sqrt(qw*qw + qx*qx + qy*qy + qz*
      qz);
15     n.qw=qw/norm; n.qx=qx/norm; n.qy=qy/norm; n.qz=qz/
      norm;
16 }

```

Блок 3. Перевод ускорения в мировую систему координат, вычитание гравитации и применение ZUPT:

Листинг 5.6. Мировое ускорение, компенсация гравитации и ZUPT

```

1  // Матрицаповоротастроки ( r00..r22) изкватерниона
2  double ax_w = r00*ax + r01*ay + r02*az;
3  double ay_w = r10*ax + r11*ay + r12*az;
4  double az_w = r20*ax + r21*ay + r22*az;
5  az_w -= (-9.81);          // вычитаемгравитацию
6
7  // ZUPT: гироскопвпокое -> мягкогасимскорость
8  double gyro_norm = std::sqrt(gx*gx + gy*gy + gz*gz);
9  if (gyro_norm < Calib::ZUPT_GYRO_THRESH) {
10     n.vx *= 0.85; n.vy *= 0.85; n.vz *= 0.85;
11     if (std::abs(ax_w) < 0.3) ax_w = 0;
12     if (std::abs(ay_w) < 0.3) ay_w = 0;
13     if (std::abs(az_w) < 0.3) az_w = 0;
14 }
15 // Двойноеинтегрирование
16 n.vx += ax_w*dt; n.vy += ay_w*dt; n.vz += az_w*dt;

```



```
17 n.px += n.vx*dt; n.py += n.vy*dt; n.pz += n.vz*dt;
```

5.2. Вычисление углов Эйлера

Завершает функцию вычисление углов Эйлера по формулам (4.8)–(4.10). Аргумент `asin` зажимается в интервал $[-1,1]$ для защиты от накопленной численной погрешности:

Листинг 5.7. Углы Эйлера из кватерниона (ZYX)

```
1 const double R2D = 180.0 / M_PI;
2 n.roll  = std::atan2(2*(qw*qx + qy*qz),
3                     1 - 2*(qx*qx + qy*qy)) * R2D;
4 double sp = std::max(-1.0, std::min(1.0, 2*(qw*qy - qz*
5   qx)));
6 n.pitch = std::asin(sp) * R2D;
7 n.yaw   = std::atan2(2*(qw*qz + qx*qy),
8                     1 - 2*(qy*qy + qz*qz)) * R2D;
```

ГЛАВА 6. ВИЗУАЛИЗАЦИЯ

6.1. Шейдеры OpenGL 3.3

Визуализатор использует две пары GLSL-шейдеров. Первая (VERT3D/) применяется для рендеринга 3D-модели полётного контроллера с диффузным освещением.

Вторая пара (VERT_FLAT/FRAG_FLAT) используется для отрисовки всех линейных примитивов без освещения: координатной сетки, мировых осей, траектории и HUD-графиков.

6.2. Процедурная модель полётного контроллера

3D-модель FC строится процедурно из параллелепипедов функцией `build_fc()`. Вспомогательная `push_box()` генерирует шесть граней куба с различными коэффициентами яркости для имитации объёма; `push_arrow()` добавляет три цветных стрелки осей (X — красная, Y — зелёная, Z — синяя):

Листинг 6.8. Процедурная сборка модели FC

```

1 static std::vector<V3> build_fc() {
2     std::vector<V3> v;
3     // Корпус -- тёмносерый- прямоугольник
4     push_box(v, 0, 0, 0, 0.8f, 0.8f, 0.14f,
5              0.13f, 0.14f, 0.19f);
6     // Белая полоска "нос" направление ( Y+)
7     push_box(v, 0, 0.37f, 0.08f, 0.7f, 0.06f, 0.04f,
8              0.95f, 0.95f, 0.95f);
9     // Боковые рёбра
10    push_box(v, 0.37f, 0, 0.08f, 0.04f, 0.8f, 0.04f,
11             0.22f, 0.22f, 0.28f);
12    push_box(v, -0.37f, 0, 0.08f, 0.04f, 0.8f, 0.04f,
13             0.22f, 0.22f, 0.28f);
14    // Стрелки осей
15    push_arrow(v, 0, 0, 0.1f, 1, 0, 0, 0.7f, 0.95f, 0.10f
16              , 0.10f); // X
17    push_arrow(v, 0, 0, 0.1f, 0, 1, 0, 0.7f, 0.10f, 0.92f

```

```

13     ,0.10f); // Y
14     push_arrow(v, 0,0,0.07f, 0,0,1, 0.7f, 0.15f,0.42f
15     ,0.98f); // Z
16     return v;
17 }

```

6.3. Главный цикл рендера

Функция `run_visualizer()` инициализирует контекст GLFW/GLEW, компилирует шейдеры, загружает геометрию в видеопамять и запускает главный цикл. Схема каждого кадра:

1. Обработать события ввода (мышь, клавиши).
2. Вычитать из `SharedBuffer` все накопившиеся пакеты, для каждого вызвать `integrate_imu()`, добавить точку траектории и обновить кольцевые буферы.
3. Обновить матрицы орбитальной камеры, следящей за FC.
4. Отрисовать **3D-сцену**: сетку, мировые оси, траекторию с цветовым затуханием, модель FC.
5. Отрисовать **HUD-панель**: четыре графика временных рядов.

Матрица модели FC формируется из текущей позиции и кватерниона ориентации с помощью GLM:

Листинг 6.9. Матрица модели из позиции и кватерниона

```

1 glm::mat4 fc_pos = glm::translate(glm::mat4(1),
2   glm::vec3((float)nav.px, (float)nav.py, (float)nav.
3   pz));
4 glm::quat q((float)nav.qw, (float)nav.qx,
5   (float)nav.qy, (float)nav.qz);
6 glm::mat4 fc_model = fc_pos * glm::mat4_cast(q);
7 glm::mat4 fc_mvp = vp * fc_model;

```

Траектория отрисовывается как набор отрезков с цветовым градиентом: старые точки темнее, новые — ярче.

6.4. HUD-панель графиков

Нижняя панель содержит четыре независимых графика на основе скользящего буфера RingBuf (300 последних значений). Каждый график отображает три сигнала (по осям X, Y, Z) красным, зелёным и синим цветами:

Таблица 6.1

Содержимое HUD-панелей

Панель	Диапазон	Сигналы
Акселерометр	$\pm 15 \text{ м/с}^2$	a_x (кр.), a_y (зел.), a_z (синий)
Гироскоп	$\pm 0,15 \text{ рад/с}$	$\omega_x, \omega_y, \omega_z$
Углы Эйлера	$\pm 180^\circ$	крен, тангаж, рыскание
Скорость	$\pm 2 \text{ м/с}$	v_x, v_y, v_z

ГЛАВА 7. ИНСТРУКЦИЯ ПО УСТАНОВКЕ И ЗАПУСКУ

7.1. Установка зависимостей

Листинг 7.10. Установка системных зависимостей

```
1 sudo apt update
2 sudo apt install build-essential cmake git \
3     libglfw3-dev libglew-dev libglm-dev libgl1-mesa-dev
```

Библиотека MAVLink является заголовочной и клонируется напрямую в каталог проекта:

Листинг 7.11. Клонирование MAVLink

```
1 cd pixhawk_imu_visualizer
2 git clone https://github.com/mavlink/c_library_v2.git
```

7.2. Сборка проекта

Листинг 7.12. Сборка через CMake

```
1 mkdir build && cd build
2 cmake ..
3 make -j$(nproc)
```

При успешной сборке в каталоге build/ появляется исполняемый файл `pixhawk_imu_reader`.

7.3. Настройка прав и запуск

Листинг 7.13. Права на порт и запуск

```
1 sudo usermod -a -G dialout $USER
2
3 sudo chmod 666 /dev/ttyACM0
4
5 cd build
6 ./pixhawk_imu_reader
```

Окно визуализации открывается немедленно. Для завершения — **Ctrl+C** в терминале.

7.4. Управление в окне визуализации

Таблица 7.1

Управление в окне визуализации

Действие	Эффект
ЛКМ + перетаскивание	Вращение камеры вокруг объекта
Колесо мыши	Приближение / отдаление (0,5 — 30 м)
Клавиша R	Сброс позиции и траектории в (0, 0, 0)
Ctrl+C	Завершение программы

ГЛАВА 8. РЕЗУЛЬТАТЫ РАБОТЫ

8.1. Внешний вид программы

На рисунке 8.1 представлен общий вид окна визуализатора в реальном времени. Верхняя область содержит 3D-сцену с моделью полётного контроллера на фоне координатной сетки и мировых осей; нижняя — четыре графика датчиков.

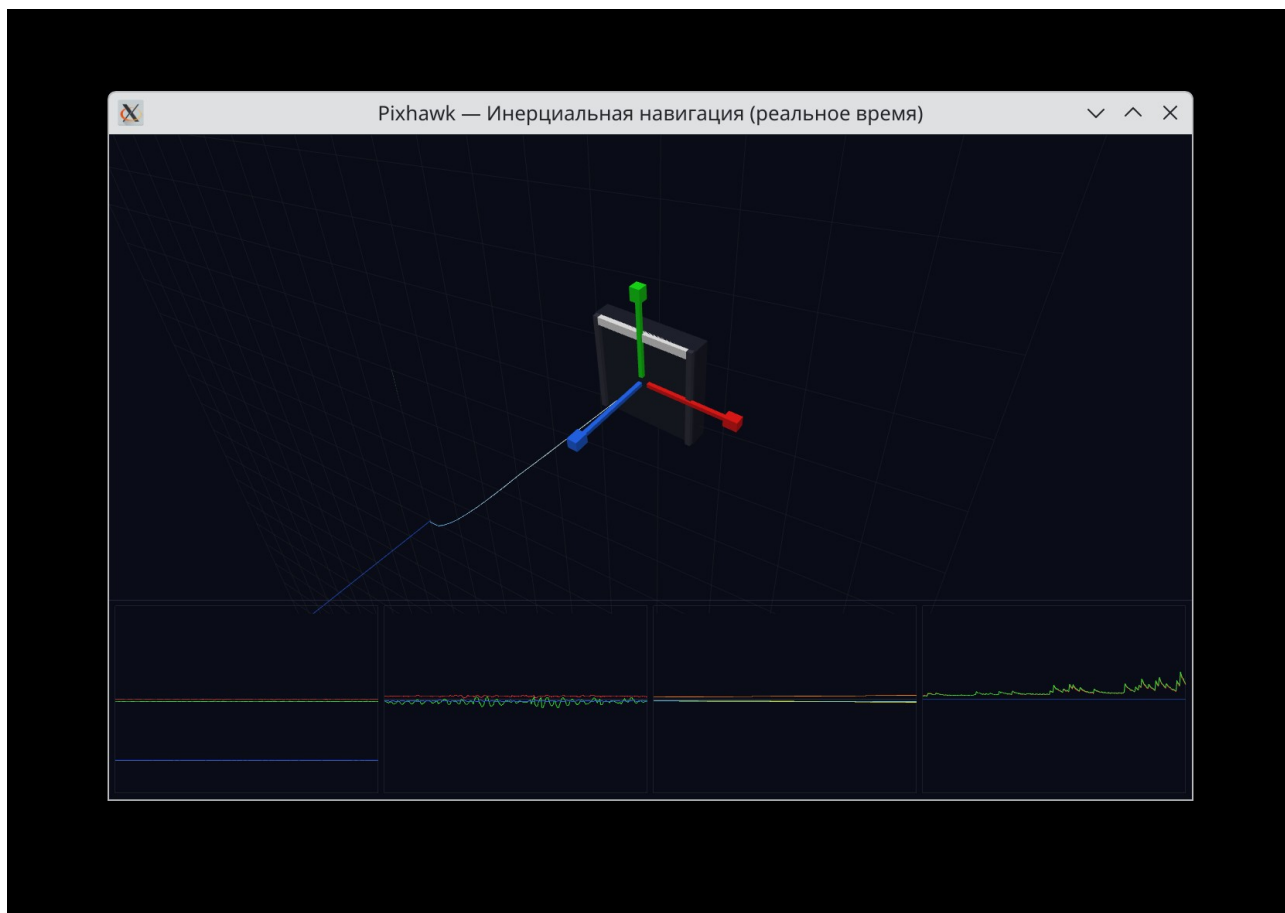


Рис. 8.1. Главное окно визуализатора: 3D-сцена и HUD-панель

На рисунке 8.2 показано состояние визуализатора при вращении полётного контроллера: 3D-модель следует за реальной ориентацией устройства, а кривые гироскопа на HUD-панели фиксируют угловые скорости по каждой из осей.

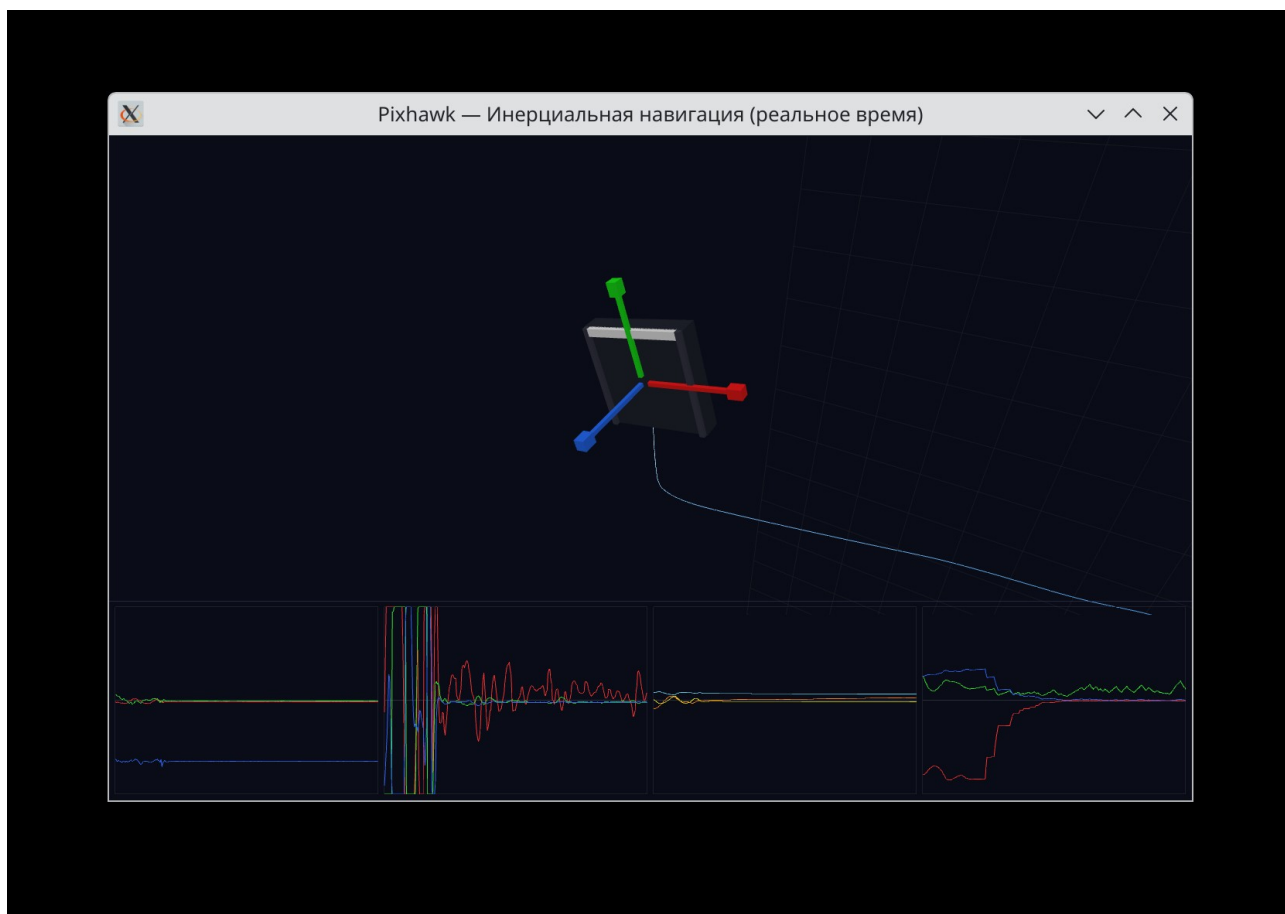


Рис. 8.2. Визуализатор при вращении устройства: реакция HUD-графиков гироскопа

На рисунке 8.3 приведён укрупнённый вид HUD-панели. Нулевые линии в каждом графике выделены серым. Видно, что в статике (покой платформы) кривые гироскопа сходятся к нулю — это подтверждает корректность подобранных смещений (4.2).

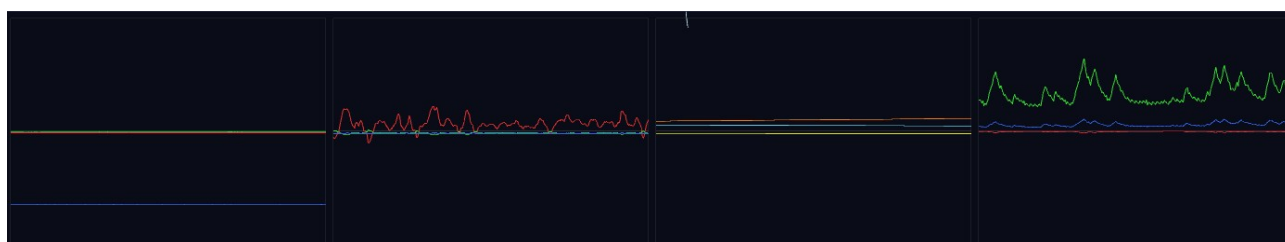


Рис. 8.3. Укрупнённый вид HUD-панели: акселерометр, гироскоп, углы Эйлера, скорость

8.2. Результаты оценки ориентации

В ходе тестирования подтверждена корректная работа алгоритма кватернионного интегрирования. При вращении полётного контроллера по всем трём

осям 3D-модель точно воспроизводила движения устройства. Углы Эйлера, вычисленные по формулам (4.8)–(4.10), соответствовали физическим поворотам.

8.3. Результаты оценки позиции

Алгоритм двойного интегрирования ускорения показал неудовлетворительные результаты при оценке абсолютных координат. Ключевая проблема — **дрейф позиции**: уже через 5–10 с накопленная ошибка достигала нескольких метров. Механизм ZUPT снизил скорость нарастания дрейфа в состоянии покоя, однако не устранил его полностью из-за ненулевого шума акселерометра порядка 10^{-3} – 10^{-2} м/с².

8.4. Анализ причин дрейфа

Пусть истинное ускорение $a(t)$ и измеренное $\hat{a}(t)$ связаны:

$$\hat{a}(t) = a(t) + b + \eta(t), \quad (8.15)$$

где b — остаточное смещение после калибровки, $\eta(t)$ — случайный шум. Ошибка позиции после двойного интегрирования за время T :

$$\delta p(T) = \frac{1}{2} b T^2 + \int_0^T (T - t) \eta(t) dt. \quad (8.16)$$

Первое слагаемое растёт *квадратично* по времени даже при малом остаточном смещении [1]. За 10 с это даёт ошибку порядка нескольких метров [7].

8.5. Возможные направления развития

Полноценная инерциальная навигация без внешних корректирующих источников принципиально невозможна [1]. Среди возможных улучшений:

1. **Фильтр Калмана с GPS-коррекцией** [1] — классический подход для авиационных INS.
2. **Фильтр Маджвика** [3] — градиентный алгоритм слияния данных гироскопа, акселерометра и магнитометра.
3. **Нелинейный дополняющий фильтр на SO(3)** [4].
4. **ZUPT с фильтром Калмана** [12] — расширенное применение метода

нулевой скорости.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы разработана система визуализации инерциальной навигации на основе полётного контроллера Pixhawk. Реализованы следующие компоненты:

- Автономный модуль первичного считывания RAW_IMU с записью данных в CSV для анализа и подбора калибровочных коэффициентов.
- Многопоточная архитектура C++14 с потокобезопасным буфером.
- Алгоритм калибровки сырых показаний с экспериментально определёнными смещениями нуля (4.2).
- Точное кватернионное интегрирование угловой скорости через ось-угол [2] с нормализацией на каждом шаге.
- Вычисление матрицы вращения $\mathbf{R} \in SO(3)$ и углов Эйлера из кватерниона.
- Двойное интегрирование ускорения для оценки позиции [1].
- Механизм ZUPT [12] для частичной компенсации дрейфа.
- OpenGL 3.3-визуализация: 3D-модель, координатная сетка, оси, траектория, HUD-панель с четырьмя графиками [8].

Алгоритм оценки ориентации показал высокое качество: 3D-модель точно воспроизводила реальные повороты устройства. Алгоритм оценки позиции продемонстрировал принципиальное ограничение МЭМС-датчиков [6]: ошибка нарастает достаточно быстро (8.16), и получение практически пригодных координат требует слияния данных IMU с корректирующими устройствами, датчиками с применением фильтра Калмана [1].

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Groves P. D. Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems. — 2nd ed. — Boston : Artech House, 2013. — 776 p.
2. Kuipers J. B. Quaternions and Rotation Sequences. — Princeton : Princeton University Press, 1999. — 398 p.
3. Madgwick S. O. H., Harrison A. J. L., Vaidyanathan R. Estimation of IMU and MARG Orientation Using a Gradient Descent Algorithm // IEEE International Conference on Rehabilitation Robotics. — 2011. — P. 1–7. — DOI: 10.1109/ICORR.2011.5975346.
4. Mahony R., Hamel T., Pflimlin J.-M. Nonlinear Complementary Filters on the Special Orthogonal Group // IEEE Transactions on Automatic Control. — 2008. — Vol. 53, no. 5. — P. 1203–1218. — DOI: 10.1109/TAC.2008.923738.
5. The Open Group. The Open Group Base Specifications Issue 7: `termios.h`. — 2018. — URL: <https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/termios.h.html> (visited on 01/01/2024).
6. Titterton D. H., Weston J. L. Strapdown Inertial Navigation Technology. — 2nd ed. — Stevenage : IET, 2004. — 558 p.
7. Woodman O. J. An Introduction to Inertial Navigation. — 2007. — URL: <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-696.pdf> (visited on 03/01/2024).
8. GLFW — An OpenGL Library. Documentation. — URL: <https://www.glfw.org/docs/latest/> (visited on 03/05/2024).
9. MAVLink Developer Guide: Common Messages. — URL: <https://mavlink.io/en/messages/common.html> (visited on 03/01/2024).
10. MAVLink: A Lightweight Message Marshalling Library for the DroneCode Ecosystem / L. Meier [et al.]. — URL: <https://github.com/mavlink/mavlink> (visited on 03/01/2024).
11. PX4 Autopilot User Guide. — URL: <https://docs.px4.io/main/en/> (visited on 03/05/2024).
12. Zero-Velocity Detection — An Algorithm Evaluation / I. Skog [et al.] // IEEE Transactions on Biomedical Engineering. — 2010. — Vol. 57, no. 11. — P. 2657–2666. — DOI: 10.1109/TBME.2010.2060723.